

# HR Ask Policy Assistant

*End of Course Project | Kang Xingyao | G2 604 349F | Section C | 3 October 2026*

## 1 Problem and intended outcome

HR-Ask is an educational prototype for a manufacturing team leader who needs policy guidance outside HR office hours. It answers questions about fictional leave, attendance and social-insurance policies and prepares a five-field request draft for later human review. The intended journey is to find a relevant rule and prepare a usable draft within three minutes. HR staff benefit when recurring questions are answered consistently and incomplete requests are identified before manual handling.

The system deliberately does not submit requests, approve leave, calculate statutory entitlements, access employee records or decide sensitive disputes. The original proposal used planning assumptions of 8–10 HR hours per week for recurring enquiries and 8–10 minutes for manual forms. These are not observations from a real employer, so neither time saving nor user benefit is claimed as measured.

I deliberately kept the first version narrow: one employee-facing question, a traceable policy answer, and an unsubmitted draft. This avoids presenting a chatbot as an HR decision-maker. The practical value I test is whether the system can make a later HR review easier by identifying relevant evidence and missing information, rather than replacing that review.

## 2 Product design and architecture

The product accepts an employee question, retrieves evidence from five versioned Markdown policy files, and classifies the request as consultation, draft, clarification or escalation. Consultation returns evidence without a ticket. An explicit draft request is parsed into `employee_id`, `request_type`, `requested_date`, `reason` and `contact_channel`. Validation keeps missing values null and reports the fields still required. Sensitive, unsupported or low-evidence requests are escalated to HR.

The local baseline uses Chinese bigrams, English tokens, aliases, TF-IDF and cosine similarity to retrieve up to three policy sections. In the OpenRouter API model mode used in the recorded demonstration, `openai/gpt-4o-mini` receives the retrieved evidence and returns answer text, citation IDs and an intent. Python retains control of field extraction and validation. The model has no tool that can execute personnel actions. This build-and-buy split keeps policy handling inspectable while renting language generation rather than training a model for five fictional documents.

I did not build an autonomous agent because the task needs no multi-step tool loop. Retrieval, intent routing and validation remain deterministic Python steps; the optional model only generates grounded language.

Course integration is explicit. Class 1 frames a bounded HR-service problem and user journey. Class 2 informs the retrieval, generation and evaluation workflow. Class 3 informs prompt repair, structured output and measured comparisons. Class 5 separates observed model inference cost from human fallback and quarterly rule-maintenance assumptions. Class 6 informs the escalation boundary, fictional data, human review and production-risk discussion. This coverage connects each design choice to a course concept while avoiding a claim that the prototype is ready for operational deployment.

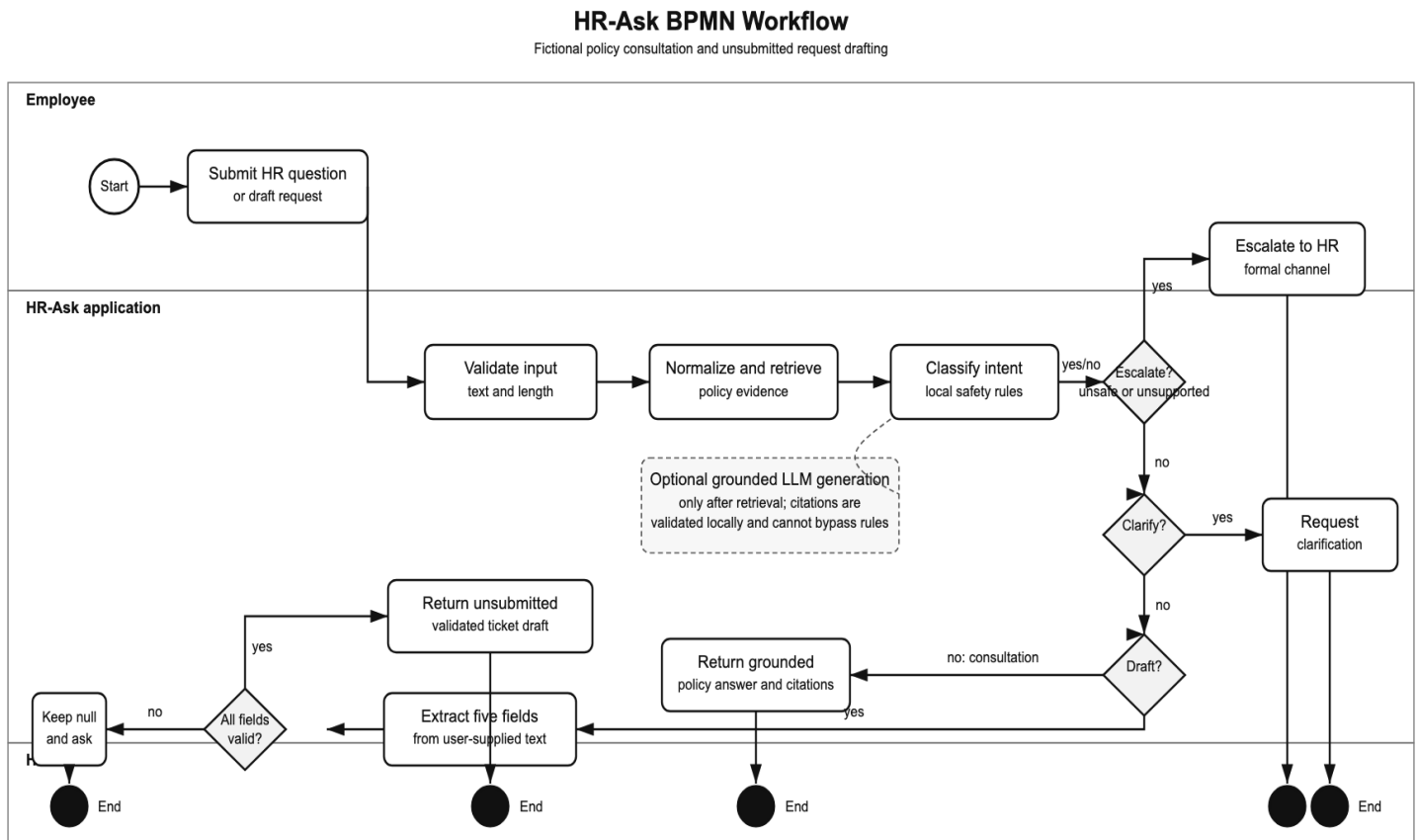


Figure 1. Evidence retrieval, safe routing and unsubmitted ticket drafting.

### 3 Evaluation design

The main development set contains 30 labelled synthetic cases across leave, attendance, social insurance and workflow boundaries. A 12-case challenge set adds ambiguity, negation, invalid dates

and unfamiliar wording. Both sets were created during development and are not independent holdouts. Results preserve source, policy and dataset hashes in the saved files. Automated checks verify routing, retrieval IDs, citation IDs, field formats and safety; they do not establish that every answer is policy-correct.

Answer quality is therefore scored separately. A retrospective AI-assisted review assessed 25 eligible saved policy answers, while I conducted a documented manual spot check of a stratified random sample of 10 answers. This documented check shows that I reviewed the sample; it is not independent HR validation. The saved live run used the revised prompt draft-intent-v2; five workflow cases were handled locally and 25 used the model without recorded API errors.

I retained prompts, the policy snapshot, labelled cases and frozen results so a reviewer can trace each reported outcome. Because the cases were designed during development, they may still favour known vocabulary and structure.

## 4 Results and critique

| Measure                     | Result         | Interpretation                                       |
|-----------------------------|----------------|------------------------------------------------------|
| Ticket structural pass rate | 8/8<br>(100%)  | Valid JSON, five fields and correct request type.    |
| First-turn complete drafts  | 5/8<br>(62.5%) | Below the proposal target of more than 85%.          |
| AI-assisted answer review   | 22/25<br>(88%) | Retrospective review of saved model answers.         |
| My manual spot check        | 8/10<br>(80%)  | Separate quality signal; not independent HR review.  |
| Engineering checks          | 30/30          | Program behaviour, not policy correctness.           |
| Informal offline diagnostic | 7/12           | Weak handling of abbreviations and informal wording. |

The strongest result is structural reliability after prompt repair. Before repair, the model interpreted drafting as execution and produced no valid drafts; after the prompt explicitly allowed an unsubmitted draft, all eight ticket-intent outputs had the required schema and type. This improvement is a before-and-after observation on one development run, not a repeated controlled experiment. Structured output and Python validation make the 8/8 result narrower than an overall success claim.

The main weakness is first-turn completion: only five of eight draft cases were complete. The three incomplete drafts safely preserved unknown fields and asked for details, but a safe incomplete draft is not a completed ticket. Answer quality also requires separate scrutiny. One observed output requested a name although all five fields were already present; another said that leave tomorrow was impossible while omitting the policy emergency route through a supervisor. The informal offline cases also missed expressions such as “tmr” and attendance wording outside the keyword gate. Correct section IDs or valid JSON do not guarantee a correct conclusion.

A superficially complete answer can still mislead an employee by missing an exception, using an overconfident tone, or routing a sensitive situation incorrectly. These results support a bounded prototype, not real-organisation policy accuracy.

## **5 Cost and operational trade offs**

The saved 25 openai/gpt-4o-mini responses averaged 666.32 input tokens, 73.68 output tokens and USD 0.000144156 provider-reported variable inference cost per query. At 400 comparable queries, inference is about USD 0.0577 per month. This is model cost per query, not total cost per successfully resolved HR case.

The service model separately includes human fallback and recurring rule maintenance. Following instructor feedback, it assumes 15–20 regex rules rewritten each quarter; the midpoint of 17.5 rules, 20 minutes per rule and USD 25 per hour gives USD 48.61 monthly maintenance. A hypothetical unsuccessful enquiry needs five HR minutes, or USD 2.0833. At useful-resolution rates of 20%, 50% and 80%, illustrative cost per useful resolution is USD 8.94, USD 2.33 and USD 0.67. Only the 80% scenario produces a positive estimated monthly value. These are transparent assumptions, not observed business results.

The cost comparison also explains why quality and escalation matter commercially. A low model-call cost does not justify deployment if an incorrect answer creates rework or sends an employee down the wrong process. A future evaluation should measure useful resolution, time saved by HR reviewers and the proportion of cases correctly escalated, using an independently reviewed policy set.

## **6 Responsible use and next steps**

The prototype is loopback-only and uses fictional data. It has no authentication, multi-turn memory, production connector, security penetration test, independent policy review or real-user study. API questions and excerpts may be sent to the selected provider, so a production system would

require access controls, retention rules, redaction, monitoring and explicit confirmation before any submission action. HR retains policy interpretation and employment decisions.

Before a pilot, the next priorities are to align answer wording with the validated draft status, collect missing fields through a guided multi-turn flow, broaden informal-language coverage, and obtain HR review of a locked policy snapshot and independent test set. These steps address the gap between successful engineering checks and trustworthy HR service outcomes.

## **7 Reproduction and deliverables**

The repository contains the policy corpus, labelled test cases, frozen evidence, source code, BPMN workflow and English demo video. The README defines the intended user, inputs, outputs, safety boundary and local setup. Repository: <https://github.com/kxy-creator/PE6201-End-of-project>

Run `python3 server.py` and open `http://127.0.0.1:8765`. Run `python3 -m unittest discover -s tests -v` for regression checks. `evaluate.py`, `compare.py` and `scripts/summarize_evidence.py` reproduce the evidence summaries. The OpenRouter API model mode used in the recorded demonstration uses a local API key that is not stored in the repository.